

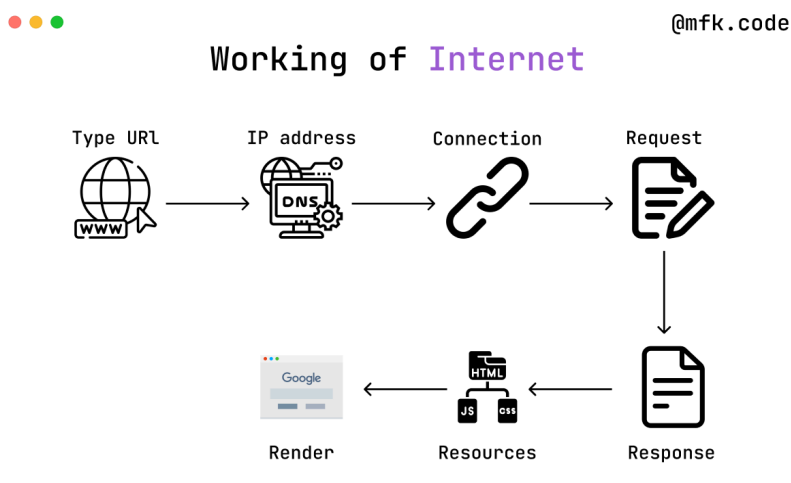
1. How the Internet Works ?

The internet works by converting a website name into an IP address (DNS), sending a request from your browser to the correct server over the internet, and returning the requested data so your browser can display the webpage.

Example :

Suppose you want to watch a video on YouTube:

1. Open your browser.
2. Type www.youtube.com.
3. DNS finds YouTube's IP address.
4. Your request travels through routers.
5. YouTube's server receives the request.
6. The server sends the webpage and video data.
7. Your browser displays the page, and the video starts playing



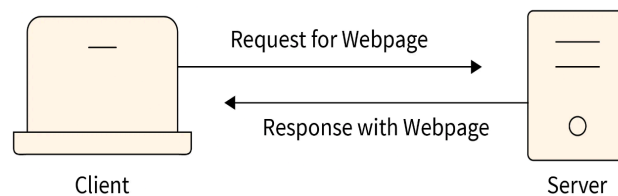
Source : [YouTube: How does the INTERNET work? | ICT #2](#)

2. Client–Server Architecture

Client-server architecture is a model where a client requests services or data, and a server provides them over a network.

Step-by-Step Process

1. Client sends a request
 - You open Chrome and type `www.youtube.com`.
 - Your browser (client) sends a request to the YouTube server.
2. Server receives the request
 - The YouTube server receives your request.
3. Server processes the request
 - The server finds the YouTube homepage and the requested video.
4. Server sends a response
 - The server sends the webpage and video data back to your browser.
5. Client displays the result
 - Your browser displays the YouTube homepage and starts playing the video.



SCALER
Topics

3. Browser

A browser is a software application that lets you access, retrieve, and display websites and web pages from the internet.

Examples of Browsers

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

4. HTTP VS HTTPS

HTTP is a protocol used to transfer data between a web browser (client) and a web server. It does not encrypt the data, so information can potentially be intercepted.

Use case: Suitable for public information where sensitive data is not being exchanged.

HTTPS is the secure version of HTTP. It uses encryption (SSL/TLS) to protect data exchanged between the browser and the server.

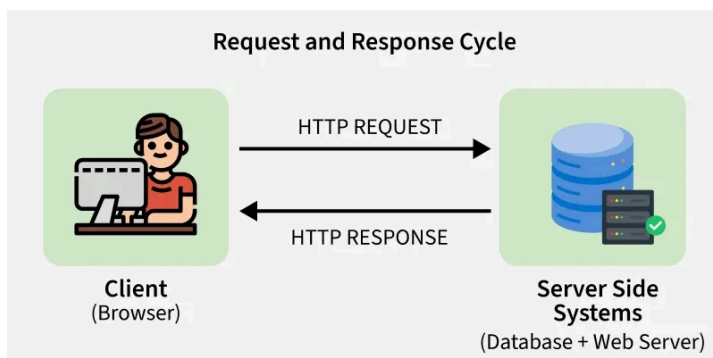
Scenario:

- You log in to your Gmail account.
- Your username and password are encrypted before being sent.
- Even if someone intercepts the data, they cannot easily read it.

Use case: Online banking, shopping, email, and any website where personal or financial information is exchanged.

5. Request-Response Cycle

The request-response cycle is the process in which a client (browser) sends a request to a server, and the server processes the request and sends back a response.



Request-Response Cycle

1. User sends a request
2. Browser creates an HTTP/HTTPS request
3. Request is sent to the web server
4. Server receives the request
5. Server processes the request

6. Server sends an HTTP/HTTPS response
7. Browser receives the response
8. Browser displays the webpage

6. Domain (Standard Explanation)

A domain (or domain name) is the human-readable name of a website that people use to access it on the internet. It represents the website's IP address in an easy-to-remember form.

For example : google.com , Facebook.com

7. DNS (Domain Name System)

DNS (Domain Name System) is a system that converts human-readable domain names into IP addresses that computers use to locate servers on the internet.

Example :

User enters google.com



DNS



Finds IP Address



Google Server



Website Opens

8. Hosting

Hosting is like renting a place on the internet where your website files are stored so people can access your website anytime.

When you create a website:

1. You create website files (HTML, CSS, images).
2. You upload them to a hosting server.
3. The hosting server stores your files.
4. Users access your website using your domain name.

Hosting Platforms

1. AWS
2. Google Cloud
3. Microsoft Azure
4. Vercel
5. Netlify
6. GitHub Pages

9. Frontend

Frontend is the client-side part of a website or application that users interact with directly. It includes everything displayed in the browser.

Technologies:

- HTML
- CSS
- JavaScript
- React
- **Examples** : Button, Text, Animation, Page layouts, Forms etc.

10. Backend

Backend is the server-side part of a website or application that handles business logic, data processing, and communication with databases. Users do not directly see the backend.

Backend Responsibilities:

- Processing requests
- User authentication
- Managing databases
- Handling application logic

Technologies:

- Node.js
- Python
- Java
- PHP

11. Full Stack

Full Stack refers to the development of both the frontend (client-side) and backend (server-side) parts of a web application. A Full Stack Developer is a developer who can work on the complete application, from the user interface to the server, database, and deployment.

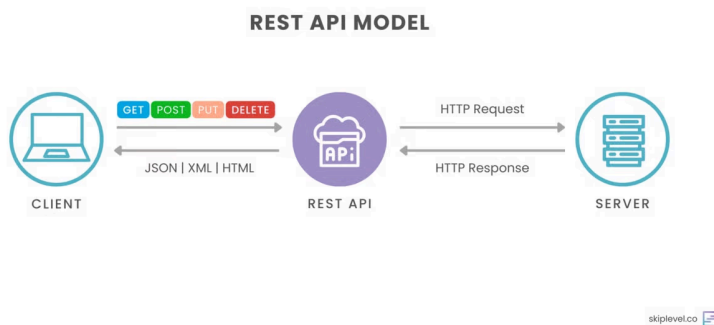
12. What is REST API?

REST API is a method that allows a client and server to communicate and exchange data using HTTP requests.

REST API Real-Life Example: Food Delivery App

1. You open a food delivery app.
2. You search for restaurants.
3. The app sends a request to the server using REST API.
4. The server finds restaurant data from the database.
5. The server sends the restaurant list back to the app.
6. The app displays restaurants on your screen.

REST API works as a bridge between the app and the restaurant server.



13. What is React?

React is an open-source JavaScript library used for building user interfaces (UI), especially for single-page web applications.

It was developed by Meta (Facebook) and is used to create fast, interactive, and reusable web interfaces.

The main concept in React is components. A component is a small, reusable piece of a user interface.

Main Features of React

1. Component-Based Architecture
2. Virtual DOM
3. Reusable Components
4. JSX Support
5. State Management
6. Fast Performance
7. Large Community Support

Advantages of React

1. Fast performance
2. Reusable components
3. Easy to learn and use
4. Large community support

Disadvantages of React

1. Frequent updates and changes
2. Requires additional libraries for complex features
3. Large applications need complex state management
4. SEO challenges in some cases
5. Too many choices can confuse developers

14. Internship Expectation

- Learn and improve frontend development skills
- Understand company workflow and coding practices
- Improve knowledge of HTML, CSS, JavaScript, and React
- Learn how to build responsive user interfaces
- Understand API integration with frontend applications
- Improve debugging and problem-solving skills
- Learn Git and version control practices
- Collaborate with developers and team members